

* By default every integral literal is of type int only*/

```
void main()
{
    byte b = 128; //error [Out of the range]
    IO.println(b);

    short s = 32768; //error [Out of the range]
    IO.println(s);
}
```

Type casting:

Type casting is nothing but converting one data type into another data type.

There are two types of type casting:

- 1) Implicit OR Widening type casting.
- 2) Explicit OR Narrowing type casting.

Implicit OR Widening type casting.

Java will automatically convert smaller data type to bigger data type. There is no data loss.

Example:
byte b = 12;
short s = b;

Explicit OR Narrowing type casting.

Converting a bigger data type into a smaller type manually. In this there is a chance of data loss.

Example:
short s = 12;
byte b = (byte) s;

```
//Assigning smaller data type value to bigger data type
void main()
{
    byte a = 12;
    short b = a; //Automatic OR WIDENING type casting
    IO.println(b);

    IO.println(".....");

    short c = 200;
    int d = c; //Automatic OR WIDENING type casting
    IO.println(d);
}

//Assigning bigger data type value to smaller data type
void main()
{
    short s = 12;
    byte b = (byte) s; //Explicit OR Narrowing
    IO.println(b);
}
```

How to find out the output in case explicit OR Narrowing casting, if the value is out of range:

byte Case:

Case 1: If the value is within the range -128 to 127 then same output will be generated

Case 2: If the value is > 127 then subtract **256** from the given number till the result value is not coming within the range

Case 3: If the value is < -128 then add 256 in the given number till the value is not coming within the range

Short type:

Case 1: If the value is within the range -32768 to 32767 then same output will be generated

Case 2: If the value is > 32767 then subtract **65536** from the given number till the value is not coming within the range

Case 3: If the value is < -32768 then add 65536 in the given number till the value is not coming within the range

//Converting bigger type to smaller type

```
void main()
{
    short s = 140;
    byte b = (byte) s; //short is converting into byte type
    IO.println(b); //-116

    short x = 127;
    byte y = (byte) x;
    IO.println(y); //127

    int p = 256;
    byte q = (byte) p;
    IO.println(q); //0

    short m = 178;
    byte n = (byte) m;
    IO.println(n); //-78

    short c = -135;
    byte d = (byte) c;
    IO.println(d); //-121

    short l = 666;
    byte o = (byte)l;
    IO.println(o); //-102

    int y1 = 33000;
    short s1 = (short) y1;
    IO.println(s1); //-32536
}
```

```
void main()
{
    byte x = (byte) 18L;
    IO.println("x value = "+x);

    long l = 29L;
    IO.println("l value = "+l);

    int y = (int) 1L;
    IO.println("y value = "+y);
}
```

Is Java pure object oriented language or not ?

No, Java is not a pure object oriented language because it is accepting **primary data type**, Actually any language which accepts primary data type is not a pure object oriented language.

Only Objects are moving in the network but not the primary data type so java has introduced Wrapper class concept to convert the primary data types into corresponding wrapper object.

Primary Data type	Wrapper Object
byte	: Byte
short	: Short
int	: Integer
long	: Long
float	: Float
double	: Double
char	: Character
boolean	: Boolean

//Wrapper classes

```
void main()
{
    Integer x = 12;
    Integer y = 24;
    Integer z = x + y;
    IO.println(z);

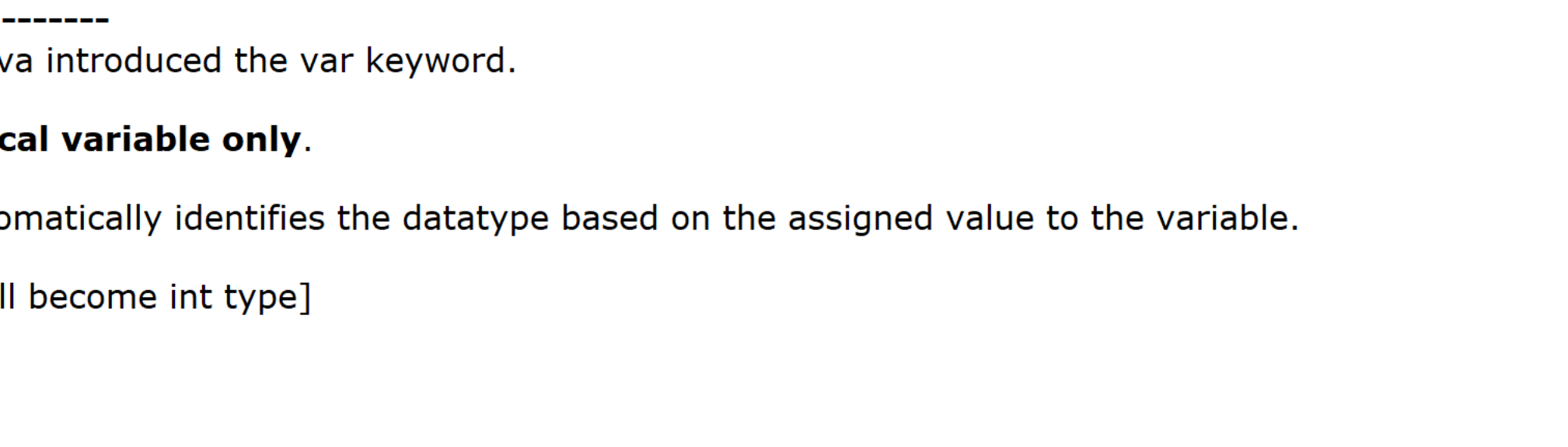
    Double d1 = 12.90;
    IO.println(d1);

    Boolean b = true;
    IO.println(b);
}
```

How to find out the minimum, maximum and size of Integral data type :

These Wrapper classes (Byte, Short, Integer, Double) provided the following final and static fields to represent the min value,max value and size of corresponding data type.

Example :
If we want to find the min value,max value and size of byte data type then **Byte class has provided the final and static fields**.



//Program to find out the range and size of Integral Data type

```
void main()
{
    IO.println("\n Byte range:");
    IO.println(" min: " + Byte.MIN_VALUE);
    IO.println(" max: " + Byte.MAX_VALUE);
    IO.println(" size : "+Byte.SIZE);

    IO.println("\n Short range:");
    IO.println(" min: " + Short.MIN_VALUE);
    IO.println(" max: " + Short.MAX_VALUE);
    IO.println(" size : "+Short.SIZE);

    IO.println("\n Integer range:");
    IO.println(" min: " + Integer.MIN_VALUE);
    IO.println(" max: " + Integer.MAX_VALUE);
    IO.println(" size : "+Integer.SIZE);

    IO.println("\n Long range:");
    IO.println(" min: " + Long.MIN_VALUE);
    IO.println(" max: " + Long.MAX_VALUE);
    IO.println(" size : "+Long.SIZE);
}
```

Providing _ (underscore) in integral Literal:

In Order to enhance the readability of large numeric literals, Java software people has provided _ from JDK 1.7V.

While writing the big numbers to separate the numbers we can use _

We can't start or end an integral literal with _ we will get compilation error.

//We can provide _ in numeric literal [JDK 1.7]

```
void main()
{
    long mobile = 98_1234_5678L;
    IO.println("Mobile Number is :"+mobile);
}
```

var keyword in java :

From Java 10, Java introduced the var keyword.

var is used for **local variable only**.

The compiler automatically identifies the datatype based on the assigned value to the variable.

Example:
var x = 12; [x will become int type]

Points to remember:

- a) Initialization is compulsory in the same line.
- b) Allowed only for local variables.
- c) Type is fixed after initialization.

//var keyword [Introduced from java 10]

```
void main()
{
    var x = 100;
    IO.println(x);

    String y = "Java";
    IO.println(y);
}
```